Digital Signal Encoder and Decoder: Specification Report

Data Communication assignment · Language: C++17 · Submission date: 21 October 2026

1. Language and libraries

- **Language:** C++17. **Compiler tested:** GCC 16.1 (MSYS2 UCRT64) on Windows 11. Any C++17 compiler (GCC ≥ 8, Clang ≥ 7, MSVC 2019) works.
- **Libraries:** only the C++ standard library (`<vector>`, `<random>`, `<functional>`, `<filesystem>`, `<fstream>`, `<cmath>`). On Windows, `<windows.h>` enables coloured console output. No external graphics library is needed.
- **Graphics:** (1) ASCII waveforms drawn in the terminal; (2) SVG plots written by the program into `output/report.html`, which any browser displays.

2. How to run

```
g++ -std=c++17 -O2 -Wall -Wextra -static -o encoder src/main.cpp
./encoder            # interactive menu
./encoder --test     # automatic self-test (3011 checks, all pass)
```

- Menu flow: *digital or analog input* → (digital: type / random / random with 4- or 8-zero runs) or (analog: shape, A, f, fs → PCM or DM) → line code → for AMI: *scrambling?* → B8ZS / HDB3 → plots → *decode?* → decoded stream and match check. Press Enter to accept the default shown in [brackets].

3. Design

File	Contents
`data_generator.hpp`	Random stream; random stream with fixed zero runs; longest palindrome (Manacher); longest zero run
`line_coding.hpp`	Abstract class `LineCoder` with one subclass per scheme (NRZ-L, NRZ-I, Manchester, Diff. Manchester, AMI, B8ZS, HDB3); waveform synthesis, CSV save/load, level measurement
`modulation.hpp`	Sampling; PCM encoder/decoder (uniform mid-rise quantizer); DM encoder/decoder; slope-overload check; SQNR
`plot.hpp` / `main.cpp`	ASCII and SVG plots, HTML report / menus, input validation, self-test

- **No if/else or switch chains:** menus are tables of (label, function) pairs. Schemes use runtime polymorphism (virtual `encode`/`decode`).
- **Decoding from the signal:** the transmitted waveform (20 samples per bit) is saved to `output/signal.csv`. The decoder reads *only* this file. It recovers the samples-per-bit from the time axis, measures the voltage at ¼ and ¾ of every bit with thresholds ±V/2, and then applies the scheme's decoding rule (level, transition, or bipolar violation).

4. Time complexity (competitive-coding part)

- **Longest palindrome:** Manacher's algorithm, **O(n)** time and O(n) memory (brute force is $O(n^2)$–$O(n^3)$). The self-test checks it against brute force on 500 random strings.
- **Zero runs for scrambling:** one right-to-left pass builds `zerosFrom[i]` (zeros starting at i). B8ZS/HDB3 then decide each substitution in O(1). Total **O(n)**. Longest zero run: one pass, O(n).
- All line codes, decoders, PCM and DM: **O(n)** in the number of bits or samples.

5. Assumptions

- Conventions follow Forouzan: NRZ-L 0 → +V, 1 → −V; NRZ-I inverts on 1; Manchester 0 = high→low, 1 = low→high (IEEE 802.3); Diff. Manchester always has a mid-bit transition, and a 0 adds a transition at the start.
- Before the first bit, the line is at +V (NRZ-I, Diff. Manchester). For AMI/B8ZS/HDB3, the "previous pulse" is taken as −V, so the first 1 is +V.
- B8ZS: 00000000 → 000VB0VB. HDB3: 0000 → 000V if the number of pulses since the last substitution is odd, else B00V.
- Pulse amplitude V = 1 V; 20 samples per bit; no noise is added to the waveform.
- PCM: uniform mid-rise quantizer over [−A, A], n bits per sample, MSB first. DM: staircase starts at 0; default $\delta = 1.2 \times$ (max slope / fs), so no slope overload.
- The "random with fixed sub-sequences" stream places the zero runs in separate slots, so they never overlap.

6. References

- B. A. Forouzan, *Data Communications and Networking*, 5th ed., McGraw-Hill, Ch. 4 (line coding, block coding, scrambling, PCM, DM).
- G. Manacher, "A new linear-time on-line algorithm for finding the smallest initial palindrome of a string", *J. ACM* 22(3), 1975.
- ITU-T Recommendation G.703: physical/electrical characteristics of hierarchical digital interfaces (HDB3, B8ZS).
- cppreference.com: C++17 standard library reference. W3C SVG 1.1 specification.